

Міністерство освіти і науки України
Центральноукраїнський національний технічний університет
Механіко-технологічний факультет
Кафедра кібербезпеки та програмного забезпечення
Дисципліна: Базові методології та технології програмування

Лабораторна робота №12

Тема: «ПРОГРАМНА РЕАЛІЗАЦІЯ АБСТРАКТНИХ ТИПІВ ДАНИХ»

Виконав: ст. гр. КН-25

Грицюк Є.В.

Перевірив: викладач

Коваленко А.С.

Кропивницький 2026

Варіант: 3

Тема: ПРОГРАМНА РЕАЛІЗАЦІЯ АБСТРАКТНИХ ТИПІВ ДАНИХ

Мета: полягає у набутті ґрунтовних вмінь і практичних навичок об'єктного аналізу й проєктування, створення класів C++ та тестування їх екземплярів, використання препроцесорних директив, макросів і макрооператорів під час реалізації програмних засобів у кросплатформовому середовищі Code::Blocks.

Концептуалізація

На зображенні — футбольний м'яч. З погляду геометрії, це ідеальна сфера. Завдання вимагає, щоб клас обчислював об'єм об'єкта за його атрибутами. Щоб знайти об'єм сфери, нам потрібен лише один фізичний параметр — радіус (або діаметр). Атрибутом нашого об'єкта буде його радіус.

Об'єктний аналіз сутності

Сутністю предметної області за варіантом завдання є об'єкт «Футбольний м'яч», який з геометричної точки зору являє собою сферу. Для програмного подання цієї абстракції розробляється абстрактний тип даних — клас мовою C++ з ідентифікатором ClassLab12_Hrytsiuk. Оскільки для ідентифікації розмірів сфери та обчислення її об'єму достатньо знати лише один базовий параметр, абстракція міститиме єдиний атрибут (дане-член) — радіус м'яча (radius). Для забезпечення принципу інкапсуляції та захисту даних цей атрибут зберігатиме значення дійсного типу (double) і буде розміщений у закритій секції класу з модифікатором доступу private.

Визначення інтерфейсів

Для взаємодії з екземпляром класу (читання, запис та обчислення похідних даних) передбачено відкритий інтерфейс у секції public, який складається з таких функцій-членів:

1. Конструктор: Забезпечує обов'язкову початкову ініціалізацію атрибута (радіуса) під час створення об'єкта в пам'яті комп'ютера.

2. Функція-аксесор (геттер): Відкрита функція (наприклад, `getRadius()`), яка повертає поточне значення закритого атрибута `radius`.
3. Функція обчислення об'єму: Відкрита функція (наприклад, `calculateVolume()`), яка виконує обчислення та повертає об'єм футбольного м'яча (сфери) за його поточним радіусом, використовуючи математичну формулу.
4. Функція-мутатор (сеттер): Відкрита функція (наприклад, `setRadius(double r)`), яка забезпечує зміну значення атрибута `radius` для вже існуючого об'єкта. Функція обов'язково реалізує валідацію вхідних даних: нове значення приймається і записується в закрите поле лише за умови, що воно є логічно коректним.

Аналіз та постановка задачі

Розробити клас `ClassLab12_Hrytsiuk`, що є програмною абстракцією сутності "Футбольний м'яч". Клас повинен містити інкапсульований (`private`) атрибут дійсного типу (`double`) для зберігання радіуса м'яча. Відкритий (`public`) інтерфейс класу має забезпечувати: початкову ініціалізацію об'єкта за допомогою конструктора, отримання поточного значення радіуса, безпечну зміну радіуса із перевіркою коректності вхідних даних (радіус має бути строго додатним числом), а також обчислення та повернення об'єму м'яча за відповідною геометричною формулою сфери.

Аналіз вимог до програмного модуля

Функціональні вимоги:

- Програмний модуль (клас) повинен зберігати стан об'єкта «Футбольний м'яч» (радіус).
- Модуль повинен забезпечувати обов'язкову ініціалізацію стану об'єкта під час його створення (через конструктор).
- Модуль повинен надавати можливість зчитувати поточне значення радіуса (геттер).
- Модуль повинен дозволяти безпечно змінювати радіус (сеттер) із вбудованою валідацією $\text{радіус} > 0$.
- Модуль повинен обчислювати та повертати об'єм сфери за формулою.

Нефункціональні вимоги:

- Модуль має бути реалізований як абстрактний тип даних (клас) мовою C++.
- Всі дані-члени повинні бути інкапсульовані (мати рівень доступу private).
- Всі операції над даними повинні здійснюватися виключно через відкритий інтерфейс (рівень доступу public).

Проектування архітектури програмного модуля

Архітектура програмного модуля будується на основі об'єктно-орієнтованого підходу. Модуль являє собою єдиний незалежний клас `ClassLab12_Hrytsiuk`, який виступає абстрактним типом даних (ADT). Клас не має складних залежностей від інших структур і взаємодіє із зовнішнім середовищем (наприклад, функцією `main` або тестовим драйвером) виключно через повідомлення (виклики відкритих методів).

Детальне проектування програмного модуля

Синтезовану абстракцію формально представлено класом C++ з таким інтерфейсом:

- Ідентифікатор класу: `ClassLab12_Hrytsiuk`
- Закриті атрибути (private):
 - `double radius`; — радіус футбольного м'яча.
- Відкриті методи (public):
 - `ClassLab12_Hrytsiuk(double r)`; — конструктор для ініціалізації радіуса (з валідацією).
 - `double getRadius()`; — функція повернення значення радіуса.
 - `void setRadius(double r)`; — функція встановлення нового значення радіуса з перевіркою ($r > 0$).
 - `double calculateVolume()`; — функція обчислення об'єму м'яча.

Назва тестового набору Test Suite Description	TS_12_1
Назва проекту / ПЗ Name of Project / Software	Hrytsiuk-task_12_1
Рівень тестування Level of Testing	Модульне тестування
Автор тестсьюту Test Suite Author	Грицюк Євгеній
Виконавець Implementer	Грицюк Євгеній

Ід-р тест-ке йса / Test Case ID	Дії (кроки) / Action (Test Steps)	Очікуваний результат / Expected Result	Результат тестування / Test Result
TC-01	Створення об'єкта: ClassLab12_Hrytsiuk ball(11.0); Виклик: ball.getRadius();	Метод повертає значення 11.0	passed
TC-02	Виклик для об'єкта з радіусом 11.0: ball.calculateVolume();	Метод повертає 5575.28	passed
TC-03	Зміна радіуса (коректне значення): ball.setRadius(5.0); Виклик: ball.getRadius();	Метод повертає значення 5.0	passed
TC-04	Зміна радіуса (некоректне значення): ball.setRadius(-3.0); Виклик: ball.getRadius();	Валідація відхиляє зміну, метод повертає попереднє значення 5.0	passed

Лістинг ModulesHrytsiuk.h

```
#ifndef MODULESHRYTSIUK_H_INCLUDED
```

```
#define MODULESHRYTSIUK_H_INCLUDED

#include <string>

double s_calculation(double x, double y, double z);

int task9_1_discount(double purchase_amount, double& discount_uah, double& final_amount);

std::string task9_2_size_converter(int ukr_size);

int task9_3_bit_counter(int n);

void task10_1_file_write(const std::string& in_filename, const std::string& out_filename);

void task10_2_file_append(const std::string& in_filename, const std::string& out_filename);

void task10_3_math_append(const std::string& out_filename, double x, double y, double z, int b);

class ClassLab12_Hrytsiuk {

private:

    double radius;

public:

    ClassLab12_Hrytsiuk(double r = 1.0);

    double getRadius();

    void setRadius(double r);

    double calculateVolume();

};

#endif
```

ЛІСТИНГ

```
#include "ModulesHrytsiuk.h"

#include <cmath>

ClassLab12_Hrytsiuk::ClassLab12_Hrytsiuk(double r) {

    if (r > 0) {

        radius = r;

    } else {

        radius = 1.0;

    }

}

double ClassLab12_Hrytsiuk::getRadius() {

    return radius;

}

void ClassLab12_Hrytsiuk::setRadius(double r) {

    if (r > 0) {

        radius = r;

    }

}

double ClassLab12_Hrytsiuk::calculateVolume() {

    const double PI = 3.14159265358979323846;

    return (4.0 / 3.0) * PI * pow(radius, 3);

}
```

Аналіз і постановка задачі завдання 2

Постановка задачі: Розробити консольний додаток Teacher, головним призначенням якого є автоматизоване модульне тестування об'єкта класу ClassLab12_Hrytsiuk. Програма повинна реалізовувати механізм контролю дотримання вимог виконання лабораторної роботи: за допомогою перевірки шляху компіляції файлу main.cpp визначати, чи знаходиться проект у регламентованій директорії \Lab12\prj. У разі виявлення порушення, програма має видати 100 звукових сигналів та записати відповідне повідомлення про помилку у файл TestResults.txt. Якщо перевірку пройдено успішно, додаток повинен зчитати тестові набори (тест-сьюти) із зовнішніх текстових файлів, створити екземпляр класу ClassLab12_Hrytsiuk, виконати тести та запроотоколювати результати (Passed/Failed) у вихідний файл звітів.

Аналіз вимог та проєктування програмного засобу Teacher

10.1. Аналіз вимог:

- Вимоги до середовища: Програма має компілюватися та виконуватися як консольний додаток C++.
- Функціональні вимоги:
 - Здобуття абсолютного шляху до файлу вихідного коду main.cpp під час компіляції (за допомогою стандартного макросу __FILE__).
 - Лексичний аналіз рядка шляху на наявність підрядка \Lab12\prj.
 - Генерація звукових сигналів (100 разів) засобами ОС або керуючим символом \a у разі помилки.
 - Створення, відкриття, дозапис та закриття файлу TestResults.txt.
- Нефункціональні вимоги: Підключення зовнішньої розробленої статичної бібліотеки (інкапсульованого класу).

Проєктування архітектури: Архітектура додатку Teacher базується на процедурному підході з використанням об'єктів. Додаток не має складного графічного інтерфейсу і працює у синхронному режимі. Логіка поділена на два основні блоки:

1. Блок верифікації середовища розробки (перевірка `__FILE__`).
2. Блок обробки даних (читання з файлів, створення об'єкта `ClassLab12_Hrytsiuk`, виконання тестів, вивід у лог). Протокол читання тест-кейсів реалізується окремою локальною функцією `runTestSuite()` для забезпечення модульності коду.

Формат тест-кейса та протокол читання

Запропонований формат текстового тест-кейсу: Для забезпечення коректного та зручного зчитування даних додатком `Teacher` з текстових файлів (тест-сьютів), кожен рядок файлу являє собою окремий тест-кейс і має таку чітку структуру, де елементи розділені пробілами:

ID_Тесту Код_Операції Вхідне_Значення Очікуваний_Результат

Де:

- ID_Тесту (тип `std::string`) — унікальний ідентифікатор контрольного прикладу (наприклад, TC-12-01).
- Код_Операції (тип `std::string`) — маркер дії, яку необхідно протестувати. Варіанти: INIT (ініціалізація), GET (отримання радіуса), SET (зміна радіуса), VOL (обчислення об'єму).
- Вхідне_Значення (тип `double`) — аргумент, який передається у відповідний метод.
- Очікуваний_Результат (тип `double`) — еталонне значення для перевірки правильності роботи методу.

Приклад рядка у файлі: TC-12-01 VOL 5.0 523.598

Протокол читання:

1. Відкриття зовнішнього текстового файлу з тест-кейсами за допомогою файлового потоку `std::ifstream`.
2. Порядкове зчитування токенів у циклі `while (file >> test_id >> action >> input_val >> expected_val)` до досягнення кінця файлу (EOF). Завдяки форматуванню через пробіл, потік автоматично парсить рядок та розподіляє дані у відповідні змінні.
3. Лексичний аналіз зчитаного Коду_Операції за допомогою умовних конструкцій `if-else`.

4. Залежно від зчитаної операції, додаток-хозяїн створює екземпляр або викликає відповідну функцію-член класу ClassLab12_Hrytsiuk.
5. Програмне порівняння фактичного результату, який повернув об'єкт, із зчитаним Очікуваним_Результатом.
6. Запис результату тестування (статуси Passed або Failed) у вихідний файл TestResults.txt.